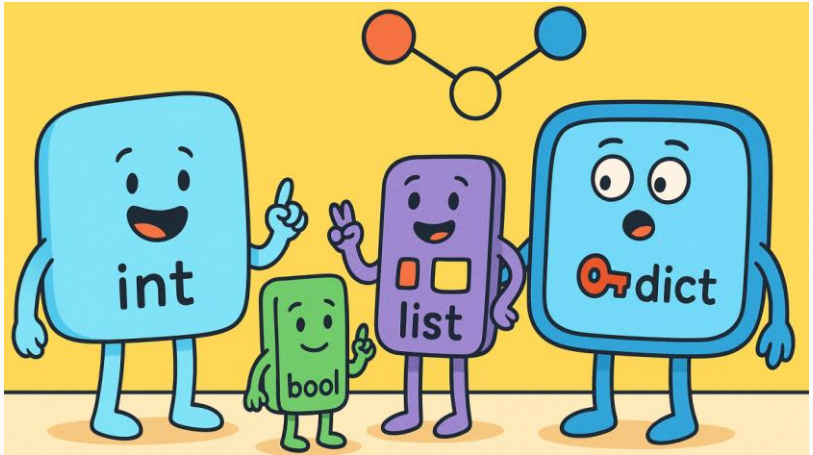
Алгоритмическая сложность. Сортировка пузырьком, бинарный поиск

Актуализация

- Что делает оператор %?
- Что такое операция swapping?
- Что делает эта операция ((shiftMain % 26) + 26) % 26?
- Как написать программу на нахождении двух уникальных числа в массиве, где не уникальные числа встречаются парами?

Алгоритмическая сложность. Big O

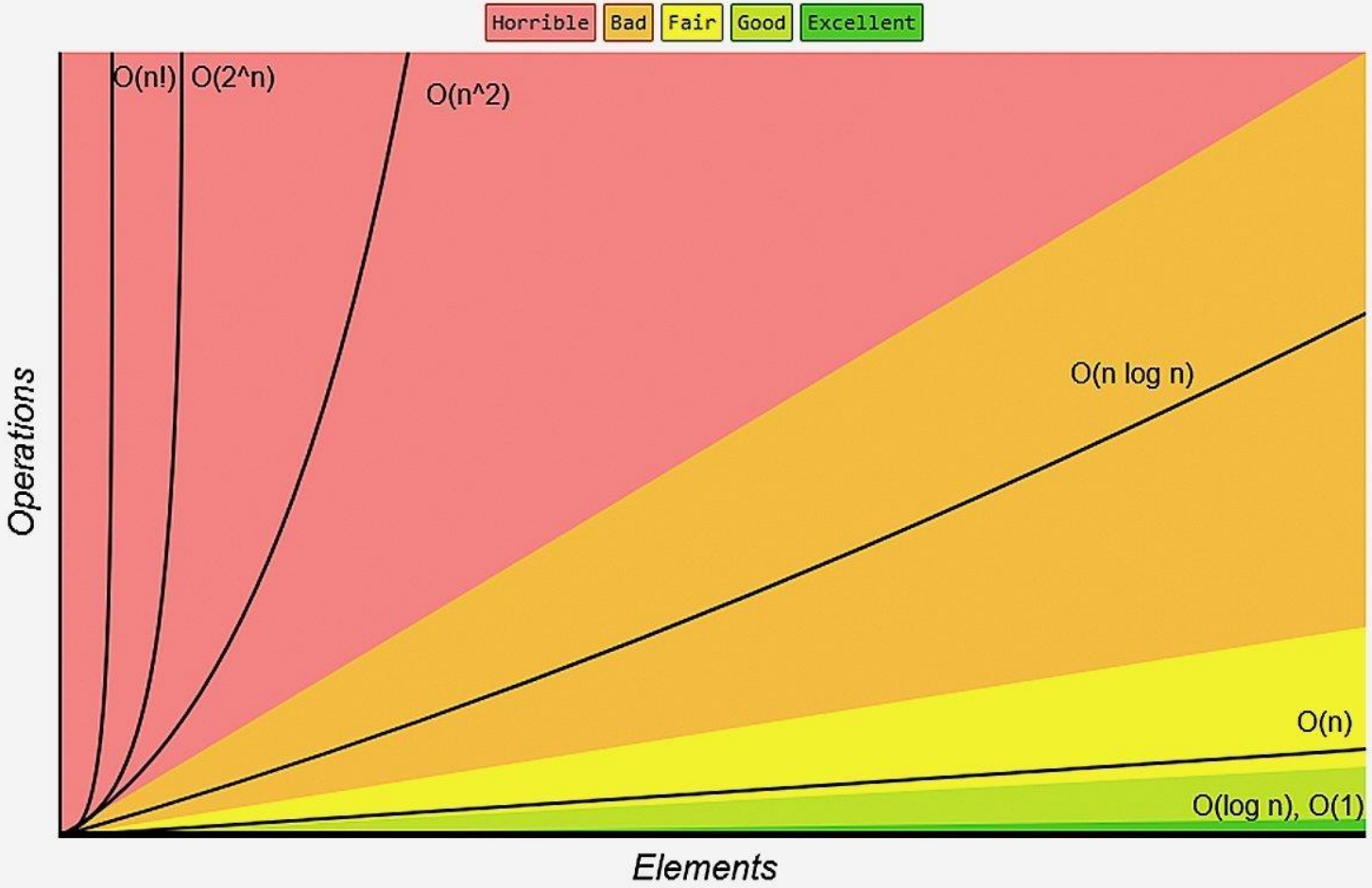
- Big O - это термин из области анализа сложности алгоритмов и структур данных в информатике. Он используется для оценки верхней границы (наихудшего случая), временной сложности алгоритма.



Алгоритмическая сложность. Big O

- **O(1): Константная сложность.** Время выполнения алгоритма не зависит от размера входных данных. Например, доступ к элементу массива по индексу.
- **O(n): Линейная сложность.** Время выполнения алгоритма пропорционально размеру входных данных. Например, просмотр всех элементов в массиве.
- **O(log n): Логарифмическая сложность.** Время выполнения алгоритма растет медленно с увеличением размера входных данных. Например, бинарный поиск в отсортированном массиве.
- **O(n^2): Квадратичная сложность.** Время выполнения алгоритма зависит от квадрата размера входных данных. Например, сортировка пузырьком (bubble sort).

Big-O Complexity Chart



O(1): Константная сложность.

- **O(1): Константная сложность.** Время выполнения алгоритма не зависит от размера входных данных. Например, доступ к элементу массива по индексу.

```
int[] arr = {1, 2, 2, 3, 5, 10, 11, 100};

int x = arr[5];           // O(1)
arr[0] = 10;              // O(1)
int last = arr[arr.length - 1]; // O(1)
```

```
int a = 90;
int b = 10;

int sum = a + b;          // O(1)
boolean even = a % 2 == 0; // O(1)
int max = a > b ? a : b; // O(1)
```

O(n): Линейная сложность

- **O(n): Линейная сложность.** Время выполнения алгоритма пропорционально размеру входных данных. Если у нас два или более последовательно идущих массива мы считаем это как O(2n), отбрасываем младшие члены и считаем как O(n)

```
int[] arr = {1, 2, 2, 3, 5, 10, 11, 100};

int s = 0;
for (int x : arr) {
    s += x;
}
```

```
for (int i = 0; i < n; i++) { /* ... */ }
for (int i = 0; i < n; i++) { /* ... */ }
```

O(n^2): Квадратичная сложность.

- **O(n^2): Квадратичная сложность.** Время выполнения алгоритма зависит от квадрата размера входных данных.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) { ... }
}
```

Вложенные циклы не всегда O(n^2):

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < 10; j++) { ... }
}
```

В этом случае будет O(n) * O(10) = O(10n) = O(n)

Пример 1. Сортировка

```java
int[] arr = {-1, 3, 4, 2, 9, 5, 4};
boolean isSorted = false;

while (!isSorted) {
    isSorted = true;
    for (int i = 0; i < arr.length - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
            isSorted = false;
        }
    }
}

System.out.println(Arrays.toString(arr));
```

Вывод:

```
[-1, 2, 3, 4, 4, 5, 9]

Process finished with exit code 0
```

Более прост в восприятии за счет флага isSorted

Пример 2. Сортировка

```
int[] arr = {100, 9, 2, 3, 5, 10, 5, -1};

for (int i = arr.length - 1; i > 0; i--) {
    for (int j = 0; j < i; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

ывод: `[-1, 2, 3, 5, 5, 9, 10, 100]`

Алгоритм отсекает после прогона конец массива, начиная с крайнего правого индекса, за счет этого немного эффективнее предыдущего варианта

Бинарный поиск

```
int[] arr = {1, 2, 2, 3, 5, 10, 11, 100};
int target = 1;
int idx = -1;
int left = 0;
int right = arr.length - 1;

while (left <= right) {
    int mid = (left + right) / 2;

    if (target < arr[mid]) right = mid - 1;
    else if (target > arr[mid]) left = mid + 1;
    else {
        idx = mid;
        break;
    }
}

System.out.println(idx);
```

1.Смотрит на средний элемент.
2.Если он не равен искомому, то понимает, в какой половине искать дальше.
3.Одну половину отбрасывает.

Важно:
Массив должен быть отсортирован

O(log n): Логарифмическая сложность

- **O(log n): Логарифмическая сложность.** Время выполнения алгоритма растет медленно с увеличением размера входных данных.

После каждого шага в бинарном поиске остается:
$n \to n/2 \to n/4 \to n/8 \to \ldots \to 1$

Сколько раз нужно разделить n на 2, чтобы остался 1 элемент?

$n / 2^k = 1$
$2^k = n$
$k = \log_2 n$

n	$\log_2 n$	Сколько шагов примерно
8	3	3
16	4	4
1024	10	10
1 000 000	≈20	≈20

В Big-O основание логарифма не важно: $\log_2 n$, $\log_{10} n$ и $\ln n$ отличаются только постоянным множителем, поэтому пишут O(log n)